

דילמות בייצוג נתונים

מיה שפירא

דילמה 1: אכיפת חוקים ב-Database או באפליקציה

אפשרות 1: אכיפה ב-Database

יתרון: הנתונים מוגנים יותר, משום שבסיס הנתונים לא מאפשר לשמור מידע לא תקין גם אם האפליקציה טועה, או אם הנתונים נכנסים ממקור אחר ולא דרכה. זה חשוב במיוחד בחוקים בסיסיים וקריטיים שחייבים להישמר תמיד.

חסרון: אכיפה ב-Database פחות גמישה, ויכולה להפוך את בסיס הנתונים למורכב יותר. לא כל חוק עסקי קל לבטא שם, במיוחד כשמדובר בחוק שתלוי בהרבה תנאים או משתנה לעיתים קרובות.

אפשרות 2: אכיפה באפליקציה

יתרון: אכיפה באפליקציה גמישה יותר, כי נוח לשנות ולעדכן את החוק בעתיד בלי לשנות את מבנה בסיס הנתונים. היא מתאימה במיוחד לחוקים עסקיים מורכבים, שתלויים בכמה תנאים או בהחלטות שמשתנות עם הזמן.

חסרון: הגנה על הנתונים פחות חזקה, משום שאם יש באג באפליקציה או אם הנתונים נכנסים ממקור אחר ולא דרך האפליקציה, עלולים להישמר בבסיס הנתונים נתונים לא תקינים.

העדפה:

הייתי מעדיפה לאכוף ב-Database חוקים פשוטים וקריטיים שאפשר להגדיר באופן חד-משמעי, כדי לשמור על אמינות הנתונים. לעומת זאת, חוקים מורכבים יותר, כמו התאמת נהג לרכב לפי גיל, סוג רישיון, ביטוח וסוג רכב, הייתי מנהלת באפליקציה. כך בסיס הנתונים שומר על שכבת הגנה בסיסית, והאפליקציה מאפשרת גמישות בחוקים העסקיים.

דילמה 2: ייצוג מסכם או מפורט

אפשרות 1: ייצוג מסכם

יתרון: ייצוג מסכם מתאים כאשר מה שמעניין את המערכת הוא בעיקר התוצאה הסופית, למשל כמה מקומות פנויים יש בחניון או כמה כלים זמינים קיימים. במקרים כאלה אין תמיד צורך להציג למשתמש את כל הפירוט, ולכן ייצוג מסכם יכול להיות פשוט, נוח ומהיר יותר לשליפה.

חסרון: קשה יותר להגן על התקינות שלו, משום שהוא חייב להתעדכן בכל שינוי רלוונטי. אם עדכון אחד מתפספס, הוא עלול להפוך ללא מדויק. בנוסף, כאשר שומרים רק את הסיכום, קשה יותר לבדוק מאיפה המספר הגיע או להבין מה גרם לטעות, כי אין מספיק מידע מפורט שמסביר את החישוב.

אפשרות 2: ייצוג מפורט

יתרון: ייצוג מפורט מאפשר לשמור את הנתונים שמהם אפשר לחשב את הנתון המסכם. למשל, בחניון אפשר לשמור את המצב של כל מקום חניה, ומתוך זה לחשב כמה מקומות פנויים יש. כך הנתון המסכם לא נשמר כמספר עצמאי בלבד, אלא מבוסס על מידע שאפשר לבדוק, לעדכן ולנתח.

חיסרון: הייצוג דורש יותר נתונים ויותר תחזוקה. במקום לשמור מספר אחד, צריך לנהל את המצב של כל מקום או כלי בנפרד. בנוסף, גם ייצוג מפורט לא תמיד מתאר את המציאות בצורה מלאה. למשל, אם רכב חונה על מדרכה, חוסם מעבר או תופס מקום בצורה לא תקינה, המערכת תדע זאת רק אם יש בה דרך לתעד מצבים כאלה. לכן הפירוט מועיל, אך גם לו יש מגבלה ברמת הדיוק.

העדפה:

הבחירה הנכונה בעיניי היא להפוך את הייצוג המפורט למקור המרכזי של הנתונים. לרוב, מה שמעניין את המערכת הוא דווקא הנתון המסכם, כמו מספר מקומות פנויים או מספר כלים זמינים. עם זאת, עדיף שהנתון המסכם יחושב מתוך מידע מפורט ולא יישמר כמספר עצמאי בלבד. כך אפשר להבין ממה הנתון נובע, לבדוק טעויות ולעדכן אותו בצורה אמינה יותר. יחד עם זאת, גם ייצוג מפורט לא מתאר את המציאות בצורה מושלמת. הוא מייצג רק את המידע שהמערכת יודעת לשמור. לכן אם רוצים שהמערכת תתייחס גם למצבים חריגים, כמו רכב שחוסם מעבר או חונה מחוץ לסימון, צריך להוסיף למודל הנתונים דרך לשמור גם את המידע הזה. אחרת, גם ייצוג מפורט של מקומות החניה לא בהכרח ישקף את החריגה.

דילמה 3: מחיר נוכחי מול מחיר היסטורי

אפשרות 1: מחיר נוכחי / מחיר בזמן התשלום

יתרון: שמירת מחיר נוכחי, כפי ששולם בפועל בזמן העסקה היא פתרון פשוט וברור יחסית. בכל עסקה נשמר הסכום שנגבה באותו רגע, ולכן אפשר לדעת מה הלקוח שילם בפועל. במערכות פשוטות, כמו עסק קטן עם עסקאות מיידיות וללא צורך בניתוחים מורכבים לאורך שנים, ייתכן שזה מספיק ואין צורך להוסיף שכבות מורכבות של שערי מטבע, הצמדות או התאמות נוספות.

חיסרון: הסכום ששולם לבדו לא תמיד מספיק כדי להבין את המשמעות הכלכלית של העסקה לאורך זמן. למשל, אם נשמר תשלום ישן של 100 לירות, בעתיד יהיה קשה להשוות אותו לעסקאות בשקלים בלי לדעת את המטבע, התקופה או שער ההמרה. גם במקרים של אינפלציה, אותו סכום יכול לייצג ערך כלכלי שונה בתקופות שונות. לכן שמירת הסכום בלבד עלולה להיות פשוטה מדי כאשר רוצים לבצע השוואות, דוחות או ניתוחים היסטוריים מדויקים.

אפשרות 2: מחיר היסטורי

יתרון: שמירת מחיר היסטורי בצורה מפורטת מאפשרת לתעד לא רק את הסכום ששולם, אלא גם את ההקשר שבו הוא נקבע: תאריך העסקה, מטבע, שער המרה, ולעיתים שם שלב בתהליך כמו הזמנה, תשלום או אספקה. כך ניתן להבין את העסקה גם בעתיד, להשוות בין תקופות, להתמודד עם מטבעות שונים ולעמוד בדרישות חשבונאיות או משפטיות בצורה מדויקת יותר.

חיסרון: ייצוג כזה מוסיף מורכבות למודל הנתונים ולתהליכי העבודה. צריך להחליט אילו נתונים נשמרים, באיזה שלב המחיר מתקבע, ואיך מטפלים במקרים שבהם התהליך נמשך זמן רב. למשל, בקניית דירה או בפרויקט גדול, יכולים לעבור חודשים או שנים בין ההזמנה, החתימה, התשלום והאספקה, ובזמן הזה המחיר, המטבע או הערך הכלכלי עשויים להשתנות משמעותית.

העדפה:

במקרה הזה הייתי בוחרת את רמת הפירוט לפי השימוש העתידי במחיר. אם מדובר בעסק פשוט עם עסקאות מיידיות, יכול להיות שמספיק לשמור בעסקה את הסכום המקורי שנרשם בזמן התשלום. כך אפשר לדעת מה היה המחיר שנגבה באותה עסקה, בלי להוסיף מורכבות מיותרת למערכת. לעומת זאת, במערכת שבה עסקאות נשמרות לאורך שנים, מתבצעות במטבעות שונים, או כוללות תהליך ארוך בין הזמנה, תשלום ואספקה, שמירת הסכום בלבד עלולה לא להספיק. במצב כזה הייתי שמורת מחיר היסטורי יחד עם פרטים שמסבירים אותו, כמו תאריך, מטבע ושער המרה במידת הצורך. כך ניתן להבין את העסקה גם בעתיד בצורה מדויקת יותר, בלי ליצור השוואה לא מדויקת בין מחיר ישן לבין מחיר נוכחי.

דילמה 4: שדה טקסט מרובה ערכים מול טבלת קשר

אפשרות 1: שדה טקסט מרובה ערכים

יתרון: שדה טקסט הוא פתרון פשוט ומהיר יותר לבנייה, כי אין צורך ליצור טבלה נוספת או להגדיר קשרים נוספים. חשוב להבחין בין שדה רגיל שמכיל ערך אחד, כמו עיר מגורים, לבין שדה שמכיל כמה ערכים יחד. כאשר מדובר בערך יחיד ופשוט, שדה רגיל יכול להיות פתרון מתאים לגמרי. גם כאשר יש כמה ערכים, שדה אחד יכול להיות נוח במקרים שבהם המידע מיועד בעיקר להצגה כללית, כמעט לא משתנה, ולא נדרש לבצע עליו חיפוש, סינון או בדיקות מורכבות.

חסרון: הבעיה המרכזית נוצרת כאשר כמה ערכים נשמרים באותה עמודה. במצב כזה העבודה עם הנתונים פחות מדויקת: חיפוש של ערך מסוים, עדכון של חלק מהרשימה ובדיקה של אחידות הערכים הופכים למסורבלים יותר. לדוגמא, אם סוגי המטוסים שטייס מוסמך להטיס נשמרים כרשימת טקסט, עלולות להופיע שגיאות כתיב או ניסוחים שונים לאותו סוג מטוס. פתרון כזה גם פחות מתאים ל-NF1, משום שכמה ערכים נשמרים באותה עמודה, בעוד שב-NF1 כל עמודה אמורה לייצג ערך אחד ברור. אם בכל זאת מחליטים לשמור כמה ערכים בשדה אחד, JSON יכול להיות מסודר יותר מטקסט חופשי, כי הוא נותן לרשימה מבנה ברור יותר. עם זאת, גם במקרה כזה עדיין מדובר בכמה ערכים בתוך שדה אחד, ולכן טיפול בכל ערך בנפרד, כמו חיפוש, עדכון או בדיקה, נשאר פחות נוח ופחות ישיר.

אפשרות 2: טבלת קשר

יתרון: טבלת קשר היא פתרון מסודר ומנורמל יותר כאשר לאותה רשומה יכולים להיות כמה ערכים באותו סוג. כל קשר נשמר כשורה נפרדת, ולכן המבנה מתאים יותר ל-NF1: במקום לשמור רשימה של כמה ערכים באותה עמודה, כל ערך נשמר באופן נפרד וברור. לדוגמא, אם טייס מוסמך להטיס כמה סוגי מטוסים, כל הסמכה יכולה להישמר כשורה נפרדת בטבלת קשר. בנוסף, ניתן להשתמש ב-Foreign Keys (מפתחות זרים) כדי לוודא שכל סוג מטוס או הסמכה שמוזנים באמת קיימים בטבלה המתאימה, ולא נשמרים כטקסט חופשי עם שגיאת כתיב או ניסוח לא אחיד.

חסרון: החיסרון הוא שהפתרון דורש מבנה מורכב יותר. צריך ליצור טבלה נוספת, להגדיר קשרים ומפתחות זרים, ולעיתים לבצע JOIN כדי לקבל את המידע המלא. כלומר, במקום שכל הערכים יופיעו

בשדה אחד, צריך לשלוף אותם דרך קשר בין טבלאות. אם מדובר במערכת קטנה מאוד, או במידע שכמעט לא משתנה ומשמש רק להצגה, ייתכן שטבלת קשר תהיה פתרון רחב מדי ביחס לצורך.

העדפה:

בדילמה זו ההבחנה המרכזית היא בין מידע שנועד רק להצגה לבין מידע שמשפיע על החלטות בפועל. כאשר מדובר בנתונים כמו הסמכות נהגים, סוגי מטוסים שטייס רשאי להטיס או מחברי טעינה, לא מדובר רק ברשימת טקסט נוחה לקריאה, אלא במידע שהמערכת צריכה להסתמך עליו. לכן במקרים כאלה הייתי מעדיפה להשתמש בטבלת קשר, שבה כל ערך נשמר בנפרד וניתן לבדיקה, חיפוש ואכיפה.

הדוגמא של רישיון טייס ממחישה את החשיבות של ההחלטה הזו. אם סוגי המטוסים נשמרים כטקסט חופשי, עלולה להיווצר טעות כתיב, חוסר אחידות או שיוך לא נכון של הסמכה. במקרה כזה הטעות אינה רק בעיה טכנית במסד הנתונים, אלא יכולה להשפיע על בטיחות ואף לסכן חיים. לכן ככל שהמידע קריטי יותר לתפעול, לאכיפה או לבטיחות, כך יש הצדקה חזקה יותר לבחור בטבלת קשר ולא בשדה טקסט מרובה ערכים.

שדה טקסט או JSON יכולים להתאים רק כאשר המידע פחות קריטי, כמעט לא משתנה, ומשמש בעיקר להצגה כללית. במקרה שבו הנתון משפיע על החלטות במערכת, טבלת קשר היא הבחירה המקצועית והבטוחה יותר.

דילמה 5: היסטוריית סטטוסים מול סטטוס נוכחי בלבד

אפשרות 1: סטטוס נוכחי בלבד

יתרון: שמירת סטטוס נוכחי בלבד היא פתרון פשוט וחסכוני יותר. היא מאפשרת לראות במהירות מה המצב העדכני של הרשומה, בלי לנהל טבלת היסטוריה נוספת. לדוגמא, ברכב אפשר לדעת מיד אם הוא זמין, מושכר או בתיקון. בנוסף, יש מקרים שבהם אחרי שהתהליך הסתיים, ההיסטוריה כבר פחות חשובה למערכת. למשל, אם הזמנה הסתיימה ללא בעיות ואין צורך לבצע עליה בדיקה חוזרת, ייתכן שהסטטוס הסופי מספיק לצורך השימוש השוטף.

חסרון: מאבדים את המידע על מה שקרה לאורך התהליך. אם שומרים רק את הסטטוס הנוכחי, אי אפשר לדעת מתי הסטטוס השתנה, כמה זמן הרשומה הייתה בכל מצב, או איפה נוצרה בעיה. למשל, במשלוח אפשר לדעת שהוא נמסר, אבל לא לדעת האם הוא התעכב בדרכו, באיזה שלב נוצר העיכוב ומתי הוא טופל.

אפשרות 2: היסטוריית סטטוסים

יתרון: שמירת היסטוריית סטטוסים מאפשרת מעקב ובקרה טובים יותר על התהליך כולו. היא מאפשרת לראות את המעברים בין הסטטוסים לאורך הזמן, לבדוק עיכובים, להבין תקלות ולשחזר מה קרה במקרה של תקלה או תלונה. מעבר לכך, שמירת המידע על התהליך חיונית כאשר רוצים לנתח אותו: למשל לבדוק באילו שלבים נוצרים עיכוב, כמה זמן לוקח לכל שלב, ואיפה אפשר לשפר את העבודה של המערכת.

חסרון: הפתרון מורכב יותר ודורש יותר מקום ותחזוקה כל שינוי סטטוס צריך להישמר כרשומה נפרדת, לרוב עם תאריך, שעה ולעיתים גם פרטים נוספים כמו מי ביצע את השינוי או מה הסיבה לשינוי. בנוסף, אם שומרים כל שינוי קטן בלי הבחנה, ההיסטוריה עלולה להיות עמוסה מדי ופחות שימושית לניתוח.

העדפה:

ההעדפה שלי היא לשמור היסטוריית סטטוסים כאשר הסטטוסים מתארים תהליך שיש חשיבות להבין ולנתח אותו גם לאחר שהוא מתקדם או מסתיים. במערכות כמו משלוחים, הזמנות, טעינות או רכבים, הסטטוס הנוכחי נותן רק תמונת מצב עדכנית, אבל הוא לא מסביר מה קרה לאורך הדרך. שמירת ההיסטוריה מאפשרת לבדוק עיכובים, להבין תקלות, לטפל בתלונות ולנתח את התהליך כדי לשפר אותו בעתיד. עם זאת, לא בכל מקרה יש צורך לשמור את כל ההיסטוריה המלאה. אם מדובר בתהליך פשוט שהסתיים ללא חריגות, ולעסק אין צורך עתידי בבקרה או בניתוח שלו, אפשר להסתפק בסטטוס הנוכחי או לשמור היסטוריה חלקית בלבד. לכן הבחירה צריכה להתבסס על השאלה האם המידע ההיסטורי מוסיף ערך אמיתי למעקב, לבקרה או לשיפור התהליך.

דילמה 6: נתון מחושב מול נתון שמור

אפשרות 1: נתון מחושב

יתרון: נתון מחושב שומר טוב יותר על תקינות הנתונים, כי הוא לא יוצר כפילות של מידע שכבר קיים במקום אחר. אם הנתון המקורי משתנה, החישוב יתבצע לפי המידע העדכני ולא לפי ערך שנשמר בעבר. לדוגמה, ממוצע דירוגים שמחושב מתוך כל הדירוגים יהיה מדויק יותר מאשר ממוצע שנשמר ולא בהכרח עודכן.

חסרון: החישוב עלול לפגוע בביצועים, במיוחד אם מדובר בחישוב כבד או כזה שמתבצע הרבה פעמים. למשל, אם בכל טעינה של עמוד צריך לחשב מחדש דירוג ממוצע מתוך הרבה רשומות, זה יכול להאט את המערכת.

אפשרות 2: נתון שמור

יתרון: נתון שמור יכול לשפר ביצועים, כי המערכת לא צריכה לחשב אותו בכל פעם מחדש אלא רק לשלוף ערך קיים. זה מתאים במיוחד לנתונים שמוצגים לעיתים קרובות או שחישובם דורש מעבר על הרבה רשומות, כמו דירוג ממוצע או מספר מקומות פנויים.

חסרון: עלול ליצור חוסר עקביות אם הוא לא מתעדכן בכל שינוי רלוונטי. אם הנתונים שעליהם הוא מבוסס משתנים, אבל הערך השמור נשאר כפי שהיה, המערכת עלולה להציג מידע שגוי. לכן צריך מנגנון עדכון ברור, אך גם מנגנון כזה אינו מבטיח תקינות מלאה. באגים בתהליך העדכון, עדכונים חלקיים או מקרים שלא נלקחו בחשבון יכולים לגרום לך שהנתון השמור לא ישקף את הנתונים האמיתיים. בנוסף, לא תמיד פשוט לוודא שהמנגנון באמת אמין בכל מצב.

העדפה:

בדילמה הזו הייתי מעדיפה שהנתונים המפורטים יהיו המקור האמין של המידע, ומהם יתבצע החישוב. כך נמנעת כפילות מיותרת ומקטינים את הסיכון לכך שערך שמור יישאר לא מעודכן. עם זאת, אם מדובר בנתון שמחושב פעמים רבות או שהחישוב שלו כבד, אפשר לשמור את התוצאה כדי לשפר ביצועים. במקרה כזה הנתון השמור צריך להיחשב כאופטימיזציה בלבד, ולא כמקור עצמאי. בנוסף, חייב להיות

מנגנון עדכון ברור, וצריך לקחת בחשבון שגם מנגנון כזה עלול להיכשל בגלל באגים או מקרים חריגים. לכן, שמירת נתון מחושב מתאימה בעיקר כאשר היתרון בביצועים מצדיק את הסיכון ואת התחזוקה הנוספת.

דילמה 7: ישות נפרדת או שדה בטבלה קיימת

אפשרות 1: שדה בטבלה קיימת

יתרון: שמירה כשדה בטבלה קיימת היא פשוטה יותר, כי אין צורך ליצור טבלה נוספת או להגדיר קשרים נוספים. כל המידע נמצא באותה טבלה, ולכן המבנה קל יותר להבנה והשימוש בו פשוט יותר. פתרון כזה מתאים כאשר מדובר במאפיין פשוט, שלא צריך לנהל עליו מידע נוסף בנפרד.

חסרון: כאשר השדה מייצג דבר שיש לו משמעות עצמאית במערכת, שדה אחד בטבלה קיימת עלול לא להספיק כדי לייצג אותו בצורה נכונה. יש מקרים שבהם אותו נתון דורש כמה פרטים משלו, ולא רק ערך אחד פשוט. למשל, אם מתייחסים לעותק מסוים של ספר, לא מספיק לשמור רק שהוא קיים; ייתכן שצריך לשמור גם מצב, מיקום וזמינות של כל עותק. במצב כזה שמירה כשדה רגיל מגבילה את המודל, מקשה על ניהול המידע, ועלולה לגרום לכך שמידע שדורש ניהול עצמאי לא יקבל ייצוג מלא ומדויק במודל הנתונים.

אפשרות 2: ישות נפרדת

יתרון: ישות נפרדת מתאימה כאשר הנתון הוא לא רק מאפיין פשוט, אלא דבר שיש לו משמעות עצמאית במערכת. ייצוג כזה מאפשר לנהל עליו מידע נפרד, לזהות אותו באופן ייחודי, לקשר אותו לנתונים אחרים, ולעקוב אחריו בצורה מדויקת יותר. בנוסף, הוא עוזר לשמור על מבנה מסודר כאשר אותו נתון מופיע במקומות רבים או דורש פרטים משלו.

חסרון: יצירת ישות נפרדת עלולה להפוך את המודל למורכב יותר אם מדובר בנתון פשוט שאין צורך לנהל אותו בפני עצמו. במצב כזה, הפירוק לטבלה נוספת לא בהכרח מוסיף ערך, אלא יכול להכביד על ההבנה והשימוש במבנה הנתונים. לכן לא כל מאפיין צריך להפוך לישות נפרדת, אלא רק כאשר יש לכך הצדקה מבחינת דרישות המערכת.

העדפה:

השאלה המרכזית כאן היא האם הנתון הוא מאפיין פשוט של רשומה קיימת, או שהוא מייצג דבר שצריך לתאר ולנהל בפני עצמו. אם מדובר בערך יחיד ופשוט, שאין לו פרטים נוספים או קשרים משלו, אפשר להשאיר אותו כשדה בטבלה קיימת. לעומת זאת, אם צריך יותר משדה אחד כדי לתאר אותו, או אם יש לו מצב, מיקום, מזהה, קשרים או היסטוריה משלו, נכון יותר לייצג אותו כישות נפרדת. לדוגמה, עותק מסוים של ספר צריך להיות נפרד מ-Book, כי לכל עותק יכולים להיות מצב, מיקום וזמינות משלו. לעומת זאת, קטגוריה פשוטה של מוצר יכולה להישאר כשדה רגיל, כל עוד לא צריך לנהל עליה מידע נוסף.

דילמה 8: קשר N:M מול מספר עמודות קבוע

אפשרות 1: מספר עמודות קבוע

יתרון: מספר עמודות קבוע יכול להיות פתרון מתאים כאשר מספר המשתתפים או הערכים מוגדר מראש ואינו צפוי להשתנות לפי דרישות המערכת. במצב כזה המבנה פשוט להבנה, משום שכל הערכים נמצאים

באותה רשומה ולא נדרשת טבלת קשר נוספת. בנוסף, עצם קיום העמודות הקבועות מגביל את מספר הערכים שניתן לשמור. למשל, אם במשחק פאדל יש תמיד ארבעה שחקנים, עמודות כמו player1, player2, player3 ו-player4 מבטיחות שלא יישמרו יותר מארבעה שחקנים באותה רשומה, ובכך מצמצמות את הצורך לאכוף בנפרד את מספר הרשומות הקשורות.

חיסרון: החיסרון הוא שמבנה כזה מגביל את הגמישות של המערכת. אם בעתיד מספר המשתתפים משתנה, או אם נדרש לשמור סוגים נוספים של קשרים, יהיה צורך לשנות את מבנה הטבלה. בנוסף, שאילתות כלליות נעשות פחות נוחות, משום שאותו סוג מידע מפוזר בין כמה עמודות שונות. לדוגמה, כדי למצוא את כל המשחקים שבהם שחקן מסוים השתתף, צריך לבדוק כמה עמודות במקום עמודה אחת אחידה. מבנה כזה גם מקשה על מניעת כפילות של אותו משתתף באותה רשומה, ועלול ליצור עמודות ריקות אם לא תמיד כל המקומות מתמלאים.

N:M קשר 2: ייצוג אפשרות

יתרון: ייצוג קשר N:M מתאים כאשר במציאות לרשומה אחת יכולים להיות כמה קשרים מאותו סוג, וכאשר הקשרים האלה חשובים מספיק למערכת. כל קשר נשמר כרשומה נפרדת, ולכן אפשר להוסיף או להסיר קשרים בלי לשנות את מבנה הטבלה. למשל, אם ספר יכול להיכתב על ידי כמה מחברים, טבלת קשר מאפשרת לשמור כל מחבר בנפרד, וגם לשמור מידע נוסף על הקשר אם צריך, כמו סדר הופעת המחברים או תפקידם. פתרון כזה מתאים במיוחד כאשר ריבוי הקשרים הוא חלק משמעותי מהמערכת ולא מקרה שולי בלבד.

חיסרון: ייצוג קשר N:M מוסיף מורכבות למודל הנתונים, כי צריך ליצור טבלת קשר, להגדיר Foreign Keys ולעיתים לבצע JOIN כדי לקבל את המידע המלא. בנוסף, לא מספיק לשאול אם קשר מרובה אפשרי מבחינה תאורטית, אלא צריך לבדוק כמה הוא נפוץ וחשוב בפועל. לדוגמה, בספרייה של בית ספר יסודי ייתכן שרוב הספרים כוללים מחבר אחד בלבד, וספרים עם כמה מחברים הם מקרה נדיר שלא משפיע הרבה על העבודה השוטפת. במצב כזה ייתכן שמודל N:M מלא יהיה פתרון מורכב מדי ביחס לצורך האמיתי של המערכת.

העדפה:

הבחירה שלי תלויה בשאלה האם הקשר המרובה הוא חלק מהותי מדרישות המערכת, או רק מקרה אפשרי אך שולי. אם המערכת צריכה לנהל קשרים שבהם מספר המשתתפים או הערכים משתנה, לבצע עליהם שאילתות, למנוע כפילויות או לשמור מידע נוסף על כל קשר, הייתי מעדיפה לייצג זאת כקשר N:M. לעומת זאת, אם במציאות מספר הערכים קבוע וברור לפי אופי הפעולה, כמו משחק פאדל שבו נדרשים ארבעה שחקנים, מספר עמודות קבוע יכול להיות פתרון פשוט וסביר יותר. גם במקרה של ספרים, אם מדובר במערכת שבה ספרים עם כמה מחברים הם חלק חשוב מהניהול והחיפוש, טבלת קשר תהיה מתאימה יותר. אבל אם זו ספרייה בסיסית, למשל בבית ספר יסודי, ורוב העבודה מתמקדת רק בזיהוי ספר לפי מחבר עיקרי, ייתכן שאין הצדקה להוסיף מורכבות של קשר N:M מלא. לכן ההחלטה צריכה להתבסס לא רק על האפשרות התיאורטית לקשר מרובה, אלא על החשיבות שלו בפועל לשימושי המערכת.

דילמה 9: כתובת מלאה מול פירוק לרכיבים

אפשרות 1: כתובת מלאה

יתרון: שמירת כתובת מלאה בשדה אחד היא פשוטה יותר להזנה ולשמירה. המשתמש יכול לכתוב את הכתובת כפי שהוא מכיר אותה, בלי שהמערכת תכריח אותו לפרק אותה למבנה קבוע. זה מתאים במיוחד כאשר הכתובת משמשת בעיקר להצגה או לקריאה אנושית, ולא לצורך ניתוחים מורכבים. בנוסף, שדה אחד יכול להתמודד טוב יותר עם כתובות חריגות שלא נכנסות בקלות לחלוקה רגילה של עיר, רחוב ומספר בית.

חסרון: קשה יותר להשתמש בכתובת לצורך חיפוש, סינון ובדיקות. אם כל הכתובת נשמרת כטקסט אחד, קשה לשלוף למשל את כל הלקוחות מאותה עיר, לבדוק כמה הזמנות הגיעו לרחוב מסוים, או לוודא שהעיר נכתבה בצורה אחידה. בנוסף, טקסט חופשי עלול ליצור חוסר אחידות, כי אותו מקום יכול להיכתב בכמה צורות שונות, ואז המערכת לא בהכרח תזהה שמדובר באותה כתובת או באותו אזור.

אפשרות 2: פירוק לרכיבים

יתרון: פירוק הכתובת לרכיבים מאפשר להשתמש בכתובת בצורה מדויקת יותר בתוך המערכת. כאשר העיר, הרחוב, מספר הבית והמיקוד נשמרים בשדות נפרדים, קל יותר לבצע חיפושים, סינונים, בדיקות ודוחות. למשל, אפשר לסנן לפי עיר, לבדוק אזורי משלוח, או לוודא ששדות חשובים לא נשארים ריקים. בנוסף, אם חלק מהרכיבים מוגדרים בצורה מסודרת, אפשר לצמצם טעויות כתיב וחוסר אחידות בנתונים.

חסרון: פירוק הכתובת מחייב להחליט מראש מהו המבנה הנכון של כתובת, וזה לא תמיד מתאים לכל המקרים. יש כתובות שאין להן רחוב רגיל, מספר בית ברור, או מבנה שמתאים לחלוקה המקובלת. אם המודל נוקשה מדי, הוא עלול לאלץ משתמשים להזין מידע בצורה לא טבעית או להשאיר שדות ריקים. לכן פירוק כתובת יכול לשפר סדר ויכולת חיפוש, אבל הוא גם עלול לפגוע בגמישות אם לא מתכננים אותו בהתאם למציאות.

העדפה:

במקרה הזה הייתי בוחרת לפי השימוש שהמערכת עושה בכתובת. אם הכתובת משמשת בעיקר להצגה או לקריאה אנושית, אפשר לשמור אותה כשדה מלא. אבל אם צריך לבצע משלוחים, סינון לפי אזור, דוחות או בדיקות תקינות, עדיף לפרק אותה לרכיבים כמו עיר, רחוב, מספר בית ומיקוד. יחד עם זאת, לא כדאי לבנות מודל קשיח מדי, כי לא כל כתובת מתאימה לחלוקה רגילה. לכן במערכת אמיתית הייתי משלבת בין רכיבים מובנים לבין שדה חופשי להשלמות או הערות, כדי לשמור גם על יכולת חיפוש וגם על גמישות.